Design Patterns

Behavioral Design Patterns

Saqif Kaisar

Department of CSE, UIU

CSE 3421 – Software Engineering

Behavioral Patterns – Overview

Purpose

Behavioral patterns describe how objects **communicate and share responsibilities**.

There are eleven behavioral patterns:

- **State**
- **Strategy**
- Interpreter
- Template Method
- Chain of Responsibility
- Command
- Iterator
- Mediator
- Memento
- Observer
- Visitor

Focus of This Lecture

State and Strategy patterns.

State Pattern

Main Idea

Allow an object to alter its behavior when its internal state changes.

Same Object $\longrightarrow$ New State $\longrightarrow$ Different Behavior

State Pattern – Method

The State pattern:

- Makes an object's behavior a function of its current state.
- Allows that behavior to change at runtime.
- Encapsulates state-specific behavior in separate state classes.
- Delegates operations to the current state through a common interface.

Context → State Interface → Current State

Key Benefit

Organize state-dependent behavior without scattering state checks throughout the context class.

State Pattern – Remote Example

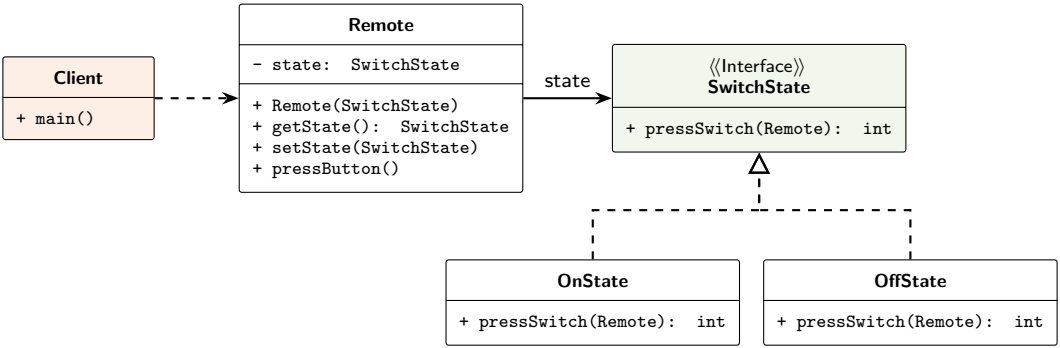
Pressing the same power button has different effects depending on the television's current state.

- **Off:** switch the television on.
- **On:** switch the television off.

State-Dependent Behavior

The remote delegates the button press to its current `SwitchState`, which performs the action and changes the state.

State Pattern – Remote UML



Delegation

`Remote.pressButton()` calls `state.pressSwitch(this)`. The concrete state can update the remote using `setState(...)`.

State Pattern – Queue Example

A service counter's behavior depends on whether it is **available** or **busy**.

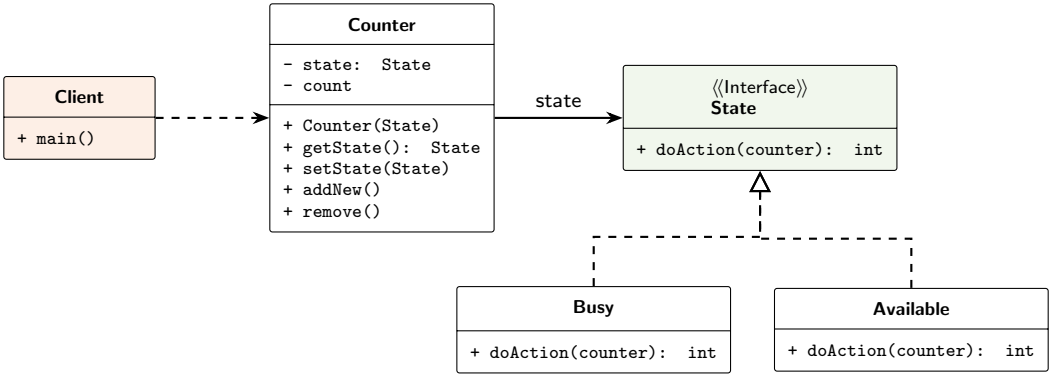
- **Available:** ready to serve a customer.
- **Busy:** already serving a customer.
- Adding or removing customers changes the queue count.



State-Dependent Behavior

The counter stores its current state and delegates state-specific actions to `Available` or `Busy`.

State Pattern – Queue UML



Roles

`Counter` is the context; `State` is the shared interface; `Busy` and `Available` provide state-specific behavior.

State Pattern – Practice Problems

Draw UML diagrams for the following scenarios using the State pattern:

1. Think of traffic lights. The red, green, and yellow colors represent different states of vehicular regulation.

2. Think of the different sound profiles on a mobile phone. You can switch between general, silent, and vibration modes.

Hint

Identify the context, its possible states, and the behavior that changes in each state. Show how the context can transition between states.

Strategy Pattern

Main Idea

Define a family of algorithms, encapsulate each one, and make them interchangeable.

Same Task $\longrightarrow$ Selected Strategy $\longrightarrow$ Result

Strategy Pattern – Method

The Strategy pattern:

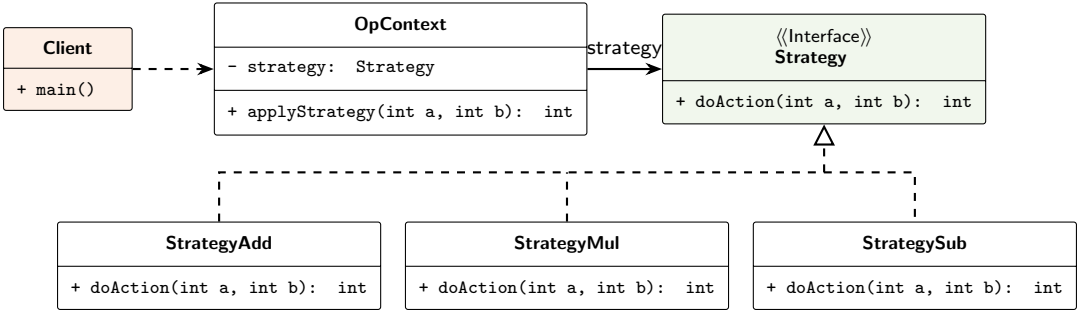
- Defines a family of algorithms behind a common interface.
- Encapsulates each algorithm in a separate strategy class.
- Allows the algorithm to be selected dynamically at runtime.
- Lets a context delegate work to the selected strategy.

Context → Strategy Interface → Selected Algorithm

Key Benefit

Change the algorithm independently of the context that uses it.

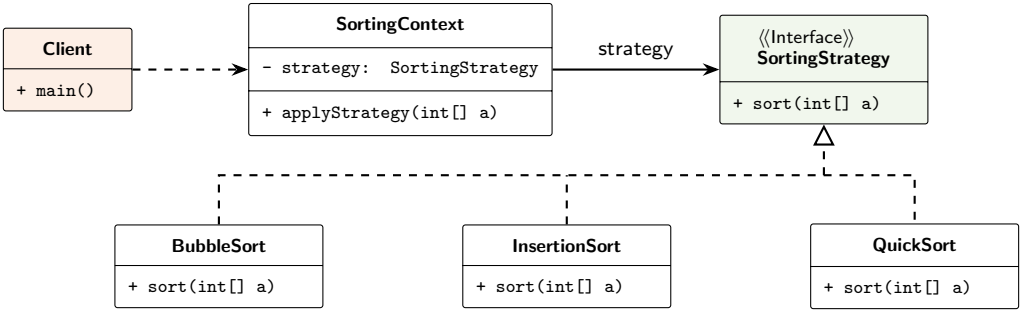
Strategy Pattern – Arithmetic UML



Interchangeable Operations

The context delegates to `strategy.doAction(a, b)`. Select addition, multiplication, or subtraction without changing the context.

Strategy Pattern – Sorting UML



One Task, Multiple Algorithms

`SortingContext` delegates to `strategy.sort(a)`. The sorting algorithm can vary while the context's public operation stays the same.

Strategy Pattern – Practice Problems

Draw UML diagrams for the following scenarios using the Strategy pattern:

1. Depending on the weather, we choose different clothing: light clothes in sunny weather, a raincoat when rain is expected, and warm, windproof clothing in winter.

2. Think of the different postures a cricket umpire uses to signal different events during a match.

Hint

Identify the shared operation, model each alternative behavior as a strategy, and show how the context uses the selected strategy.

Thank You

Questions?